Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Інститут комп'ютерних систем
Кафедра інформаційних систем

Сучков Олександр Ігорович,
студент групи АІ-221

ЗВІТ

з виробничої практики

Спеціальність:

122 Комп’ютерні науки

Освітня програма: Комп’ютерні науки

Керівник від НУОП

к.т.н., доцент
(науковий ступінь, вчене звання)

Бабілунга О.Ю.
(прізвище та ініціали) (підпис)

«27» лютого 2026 р.

Керівник від

ТОВ «ІТ-КАРКАС»
(назва підприємства, організації, установи)

(прізвище та ініціали) (підпис)

«27» лютого 2026 р.

Одеса – 2026

ЗМІСТ

ВСТУП 5

1 ОРГАНІЗАЦІЙНА, ФУНКЦІОНАЛЬНА ТА СТРУКТУРНА ХАРАКТЕРИСТИКА ПІДПРИЄМСТВА 6

1.1 Загальні відомості про компанію ТОВ «ІТ-КАРКАС» 6

1.2 Аналіз методологій розробки в умовах компанії 6

Висновки до розділу 1 7

2 ІНДИВІДУАЛЬНЕ ЗАВДАННЯ ЗА ТЕМОЮ «РОЗРОБКА ЗАСТОСУНКУ ОЦІНКИ ВИКОНАННЯ ВПРАВ ГІМНАСТИКИ ЙОГА З ВИКОРИСТАННЯМ НЕЙРОМЕРЕЖІ MEDIAPIPE» 8

2.1 Огляд існуючих аналогів застосунків для заняття йогою 8

2.1.1. Порівняння аналогів 8

2.1.2. Вибір архітектурного шаблону 12

2.1.3. Вибір мови програмування 13

2.1.4. Вибір середовища програмування 14

2.1.5. Вибір системи управління базами даних 16

2.1.6. Вибір фреймворків та бібліотек 17

2.1.7. Вибір патернів проєктування 18

2.2 Проєктування застосунку підтримки виконання вправ гімнастики йога 19

2.2.1. Мета та задачі. 19

2.2.2. Типи користувачів. 20

2.2.3. Функціональні вимоги 20

2.2.4. Нефункціональні вимоги 21

2.2.5. Ідентифікація архетипу 22

2.2.6. Уявлення інтерфейсу користувача 23

2.2.7. Логічне уявлення 24

2.2.8. Уявлення розгортання 25

2.2.9. Уявлення шарів 25

2.2.10. Уявлення процесів 26

2.2.11. Уявлення даних 26

2.2.12. Уявлення комунікацій 27

2.2.13. Опис стеку технологій 28

2.3 Результат роботи 29

Висновки до розділу 2 31

ВИСНОВКИ 33

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 34

ДОДАТОК 1 – ПОВНИЙ ПРОГРАМНИЙ КОД YOGATRAINERAPP 36

ДОДАТОК 2 – ПОВНА JSON-СТРУКТУРА ВПРАВ ДЛЯ ПРОГРАМИ 45

ДОДАТОК 3 – ПОВНА ЗАПОВНЕНА ДЛЯ ВПРАВИ «ВОЇН ІІ» СТРУКТУРА 47

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ
на переддипломну практику

здобувачу вищої освіти Сучкову Олександру Ігоровичу
(прізвище, ім'я, по батькові)

номер групи АІ221 , номер моб. телефона _____
e-mail: 10252304@stud.op.edu.ua

Тема роботи Розробка застосунку оцінки виконання вправ гімнастики йога з використанням нейромережі Mediapipe

Керівник роботи Блажко Олександр Анатолійович, к.т.н., доцент
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

ЗАВДАННЯ за темою кваліфікаційної роботи

№ з/п	Назва етапів кваліфікаційної роботи	Строк виконання	Примітка
1.	Огляд існуючих аналогів: пошук аналогів, порівняння аналогів, вибір архітектурного шаблону, вибір мови програмування, вибір середовища програмування, вибір системи управління базами даних, вибір фреймворків та бібліотек, вибір патернів проєктування.	02.02.2026 – 08.02.2026	
2.	Проєктування: визначення мети та задач, визначення типів користувачів, визначення інформаційних потоків, визначення функціональних вимог, моделювання прецедентів, визначення нефункціональних вимог, ідентифікація архетипу, уявлення інтерфейсу користувача, логічне уявлення, уявлення розгортання, уявлення шарів, уявлення процесів, уявлення даних, уявлення комунікацій, опис стеку технологій.	09.02.2026 – 19.02.2026	
3.	Реалізація та тестування: реалізація БД, реалізація програмного модуля редагування БД, тестове заповнення БД. Приклади опису етапів – https://bit.ly/4a5UJjq	20.02.2026 – 27.02.2026	

Керівник кваліфікаційної роботи _____ Блажко О.А.
(підпис) (прізвище, ініціали)

Здобувач _____ Сучков О.І.
(підпис) (прізвище, ініціали)

Дата видачі завдання 30.01.2026

Керівник практики від університету _____
_____
(підпис) (прізвище, ініціали)

ВСТУП

Виробнича практика була виконана у ТОВ «ІТ-КАРКАС» - ІТ-компанії, що спеціалізується на розробці програмного забезпечення та наданні ІТ-послуг. Під час практики було прослухано лекцію, де були розглянуті сучасні підходи до розробки програмного забезпечення.

Метою переддипломної практики у ТОВ «ІТ-КАРКАС» було ознайомлення з процесами розробки програмного забезпечення у реальному виробничому середовищі. У результаті практики було поглиблено теоретичні знання у галузі програмної інженерії.

Додатково, було виконано індивідуальне завдання за темою «Розробка застосунку оцінки виконання вправ гімнастики йога з використанням нейромережі Mediapipe».

Під час виконання індивідуального завдання було розроблено вдосконалену програму-тренер з поз йоги. На відміну від попередньої версії, нова програма здійснює точне вимірювання кутів суглобів тіла за допомогою тригонометричних обчислень, підтримує два режими роботи (практика та тренер), а також дозволяє зберігати та завантажувати конфігурації вправ у форматі JSON.

1 ОРГАНІЗАЦІЙНА, ФУНКЦІОНАЛЬНА ТА СТРУКТУРНА ХАРАКТЕРИСТИКА ПІДПРИЄМСТВА

1.1 Загальні відомості про компанію ТОВ «ІТ-КАРКАС»

Компанія ТОВ «ІТ-КАРКАС» є ключовим об’єктом дослідження під час проходження виробничої практики. Підприємство зареєстроване та провадить свою основну діяльність у місті Одеса (вул. Тираспольська, 27/29), а також має представництво у місті Київ.

Компанія ТОВ «ІТ-КАРКАС» займається розробкою програмних рішень для бізнесу та надає послуги з ІТ-консалтингу. Окрім розробки ПЗ, підприємство позиціонує себе як потужна телекомунікаційна компанія, що надає послуги інтернет-доступу, організацію транспортних каналів передачі даних та оренду оптоволоконних ємностей по території України. Будучи учасником точок обміну трафіком, таких як DTEL-IX та UA-IX, компанія володіє експертизою в архітектурі мереж та складних системних рішеннях.

Діяльність ТОВ «ІТ-КАРКАС» торкається кількох значних секторів сучасного ІТ-ринку, таких як розробка спеціалізованого програмного забезпечення, ІТ-консалтинг та аудит, системна інтеграція та кібербезпека, та робота з хмарними середовищами.

1.2 Аналіз методологій розробки в умовах компанії

Під час практики було прослухано лекцію, присвячену роботі компанії та сучасним підходам до організації процесу розробки програмного забезпечення. Було детально розібрано життєвий цикл розробки ПЗ, прийнятий в ТОВ «ІТ-КАРКАС». Компанія використовує ітеративний підхід, де кожен етап жорстко регламентований. Це необхідно для забезпечення стабільності сервісів, якими користуються сотні корпоративних клієнтів. Обговорювалась тема організації командної роботи: використання системи контролю версій Git, ведення гілок розробки (GitFlow), процедури код-рев'ю та документування коду. Детальний розгляд моделі GitFlow дозволив зрозуміти, як розподіляються ролі в команді. Ця модель роботи є важливою

для «IT-КАРКАС», оскільки компанія обслуговує діючі мережі зв'язку, де проблеми програмного забезпечення можуть приводити до простою критичної інфраструктури.

Доповідачі поділились практичним досвідом роботи з pull-request'ами та стандартами написання commit-повідомлень. Було наголошено, що Code Review, окрім застереження проблем із програмним кодом, також дозволяє обмін знаннями всередині команди. Також було приділено увагу стандартизації формату комітів. Втім, більшість з цього було раніше вивчено у другому семестрі першого курсу як частину навчального матеріалу з дисципліни «Операційні Системи».

Лекція надала розуміння того, як теоретичні знання з університетських курсів застосовуються у реальних умовах комерційної розробки. Під час навчання, основний акцент робився на синтаксисі команд Git та базових операціях у терміналі. На практиці у лекції ТОВ «IT-КАРКАС» фокус змістився на стратегічне планування змін, вирішення merge conflicts у великих репозиторіях та інтеграцію Git із системами CI/CD.

Висновки до розділу 1

Я зміг порівняти академічний підхід до програмування з виробничим та ознайомитись зі стандартами, прийнятими у сучасних IT-командах. Отримані знання стали важливим доповненням до моєї теоретичної бази. Розуміння структури компанії допомогло мені отримати іншу перспективу на роботу у великих IT-структурах, хоч це і не допомогло мені у виконанні індивідувального завдання, оскільки воно жодним чином не пов’язано з такими структурами.

2 ІНДИВІДУАЛЬНЕ ЗАВДАННЯ ЗА ТЕМОЮ «РОЗРОБКА ЗАСТОСУНКУ ОЦІНКИ ВИКОНАННЯ ВПРАВ ГІМНАСТИКИ ЙОГА З ВИКОРИСТАННЯМ НЕЙРОМЕРЕЖІ MEDIAPIPE»

2.1 Огляд існуючих аналогів застосунків для заняття йогою

2.1.1. Порівняння аналогів

У сучасному світі технологічного прогресу та цифрових інновацій важливою складовою будь-якого дослідження чи розробки є аналіз існуючих аналогів. Вивчення вже наявних рішень дозволяє не лише отримати глибше розуміння предметної області, але й визначити ключові тенденції, сильні та слабкі сторони продуктів, що використовуються на ринку. Це дає змогу обґрунтовано приймати рішення щодо подальшого розвитку, уникаючи можливих помилок і дублювання вже створених технологій.

Аналіз аналогів допомагає зрозуміти, які функціональні можливості, архітектурні підходи та користувацький досвід пропонують існуючі продукти у сфері автоматизованого контролю фізичної активності. Завдяки цьому можна визначити, які особливості варто адаптувати чи покращити у власній розробці. Крім того, такий аналіз сприяє оцінці конкурентоспроможності нових рішень та виявленню ніш, які ще не заповнені, зокрема в аспектах гнучкого налаштування вправ користувачем.

У даній роботі буде розглянуто існуючі аналоги за відповідною тематикою, проведено їхню оцінку з точки зору функціональності, зручності використання, продуктивності та інших важливих характеристик.

Таким чином, огляд існуючих аналогів є невід'ємною частиною дослідження, що забезпечує всебічний аналіз ринку та сприяє прийняттю ефективних рішень у процесі розробки та впровадження інноваційних технологій у сфері спортивних застосунків.

Застосунок Daily Yoga.

Daily Yoga – один із найпопулярніших мобільних застосунків для практики йоги, що пропонує понад 500 поз та більше 200 структурованих програм тренувань. Застосунок доступний для платформ iOS та Android і орієнтований на широку аудиторію – від початківців до досвідчених практиків. Інтерфейс Daily Yoga відрізняється зручною навігацією, а бібліотека поз супроводжується детальними відеоінструкціями та анімаціями.

Основний функціонал Daily Yoga включає покрокові відеоуроки з демонстрацією правильного виконання вправ, можливість складати власні програми тренувань, відстеження прогресу та планування занять. Застосунок підтримує як самостійне, так і кероване виконання вправ з голосовим супроводом. Він також має функцію медитацій та дихальних практик, що виходить за межі суто фізичних вправ.

Однак Daily Yoga не має функції розпізнавання поз через камеру. Застосунок не може автоматично оцінити правильність виконання вправи конкретним користувачем – контроль якості виконання покладається виключно на самого користувача. Крім того, база поз є фіксованою та не допускає додавання власних вправ тренером. Повний функціонал застосунку доступний лише за передплатою.

Застосунок Kemtai.

Kemtai – веб-застосунок для фітнесу нового покоління, що використовує штучний інтелект та комп'ютерний зір для аналізу рухів користувача в реальному часі через звичайну веб-камеру. Застосунок орієнтований на персональні тренування без тренера, надаючи автоматичний зворотний зв'язок щодо правильності виконання вправ. Kemtai використовує технологію відстеження суглобів для розрахунку кутів між кінцівками та порівняння їх з еталонними значеннями.

Ключовою перевагою Kemtai є саме наявність відстеження поз через камеру з автоматичною оцінкою правильності виконання вправ. Система підраховує кількість повторень, вимірює кути суглобів та надає голосові підказки у процесі тренування. Це суттєво відрізняє застосунок від більшості конкурентів, що пропонують лише пасивні відеоінструкції.

Разом з тим, Kemtai має суттєві обмеження. Застосунок є веб-орієнтованим та вимагає постійного підключення до Інтернету, що унеможливлює офлайн-використання. Бібліотека вправ є повністю фіксованою – тренер не може додати власні вправи з індивідуальними кутовими параметрами. Повний функціонал застосунку доступний виключно за платною передплатою, що обмежує його доступність. Крім того, застосунок зорієнтований переважно на загальні фітнес-вправи, а не конкретно на йогу.

Застосунок Down Dog.

Down Dog – мобільний застосунок для практики йоги, що відрізняється від конкурентів можливістю генерувати унікальні програми тренувань на основі вподобань користувача. Застосунок пропонує понад 60 000 різних комбінацій занять, адаптованих під рівень підготовки, тривалість та стиль йоги. Down Dog доступний для iOS та Android і підтримує аудіосупровід від реальних інструкторів.

Застосунок відзначається гнучкістю в налаштуванні тренувань: користувач може обирати тривалість заняття, рівень складності, музичний фон та мову голосового супроводу. Є функції для різних стилів йоги – Vinyasa, Hatha, Yin, Restorative тощо. Down Dog підтримує офлайн-режим після завантаження контенту.

Проте Down Dog, як і Daily Yoga, не має функції відстеження поз через камеру. Правильність виконання вправ не перевіряється автоматично. База вправ також є фіксованою – тренер не може додати власні пози з

індивідуальними параметрами. Повний доступ до функціоналу вимагає платної передплати.

Результати порівняння.

Проведений аналіз трьох застосунків-аналогів дозволяє виявити їхні спільні слабкі місця. Daily Yoga та Down Dog є потужними інструментами для надання інструкцій з виконання вправ, проте жоден з них не здатен автоматично оцінити правильність виконання пози конкретним користувачем. Kemtai вирішує цю проблему за рахунок відстеження рухів, але є виключно хмарним рішенням та не дозволяє тренеру визначати власні вправи. Жоден з розглянутих аналогів не надає тренерам інструментарію для самостійного визначення кутових параметрів нових поз.

Порівняння функціоналу розглянутих застосунків подано в табл. 1.1.

Як видно з табл. 1.1, власна розробка поєднує ключові переваги існуючих аналогів та усуває їхні основні недоліки: надає відстеження поз через камеру з точним вимірюванням кутів суглобів, дозволяє тренерам записувати власні вправи без програмування, підтримує повноцінну офлайн-роботу та є повністю безкоштовним рішенням з відкритим кодом.

Таблиця 1.1 – Порівняння функціоналу аналогів застосунків для йоги

Функціонал	**Daily Yoga**	**Kemtai**	**Down Dog**	**Власна розробка**
Відстеження поз через камеру	–	+	–	+
Автоматична оцінка правильності пози	–	+	–	+
Можливість тренеру додавати власні пози	–	–	–	+

Налаштування кутових параметрів вправ	–	–	–	+
Покрокове виконання вправ	+	+	+	+
Звукові сповіщення	+	+	+	+
Офлайн-режим	Частково	–	Частково	+
Безкоштовне використання	Частково	–	Частково	+

2.1.2. Вибір архітектурного шаблону

Вибір архітектурного шаблону є фундаментальним рішенням, яке визначає загальну структуру системи, спосіб взаємодії між компонентами та підходи до її розширення. При виборі архітектурного шаблону для даного проєкту були розглянуті основні підходи: монолітна архітектура, мікросервісна архітектура та клієнт-серверна архітектура.

Мікросервісна архітектура передбачає розподіл системи на незалежні сервіси, кожен з яких відповідає за окрему функцію. Такий підхід забезпечує високу масштабованість і незалежне розгортання компонентів, проте є надлишково складним для локального десктопного застосунку, де всі компоненти виконуються в одному процесі на одній машині.

Клієнт-серверна архітектура підходить для веб-застосунків та систем із розподіленим доступом. Оскільки розроблювана програма є локальним десктопним застосунком без серверної частини та без потреби в мережевій взаємодії, даний підхід не є оптимальним.

Монолітна архітектура обрана як основний архітектурний шаблон для даного проєкту. Застосунок реалізовано як єдиний Python-процес, де всі модулі (захоплення відеопотоку, розпізнавання поз, управління базою даних, відображення інтерфейсу) тісно інтегровані. Монолітна архітектура

є оптимальним вибором з огляду на: локальний характер роботи застосунку без потреби в мережевій взаємодії; відсутність вимог до горизонтального масштабування; простоту розгортання – достатньо одного Python-інтерпретатора та необхідних бібліотек; зручність розробки та налагодження в середовищі PyCharm. Всередині монолітної архітектури реалізовано чіткий поділ на логічні модулі відповідно до принципу єдиної відповідальності.

2.1.3. Вибір мови програмування

Вибір мови програмування є одним із ключових рішень, оскільки він визначає доступні бібліотеки, продуктивність обробки відеопотоку, зручність розробки та можливості інтеграції з нейромережевими інструментами. Для проєкту були розглянуті мови, що підтримують роботу з комп'ютерним зором та обробкою відеопотоку в реальному часі.

C++ забезпечує найвищу продуктивність та підтримується бібліотеками OpenCV і MediaPipe на нативному рівні. Проте C++ вимагає значно більших витрат часу на розробку через ручне управління пам'яттю, відсутність зборщика сміття та складнішу систему збірки. Для прототипного проєкту індивідуальної розробки ця складність є невиправданою.

JavaScript (Node.js з Electron) теоретично дозволяє створювати кросплатформні десктопні застосунки. Однак MediaPipe не має офіційної підтримки для Node.js, а бібліотеки комп'ютерного зору в екосистемі JavaScript значно поступаються Python-аналогам за повнотою функціоналу та якістю документації.

Java надає хорошу кросплатформність та зрілу екосистему, проте підтримка MediaPipe для Java не є офіційною та потребує JNI-обгорток. Бібліотека OpenCV має Java-binding, але він є менш повним порівняно з Python-версією. Крім того, Java вимагає більше шаблонного коду, що ускладнює швидку розробку.

Python [10] обраний як основна мова програмування з таких причин: MediaPipe [7] та OpenCV [8] мають першокласну підтримку саме для Python з найбільш повною та актуальною документацією; Python має найбагатшу екосистему бібліотек для машинного навчання, комп'ютерного зору та обробки даних; висока швидкість розробки завдяки динамічній типізації та виразному синтаксису; бібліотека NumPy забезпечує ефективні матричні обчислення для обробки координат суглобів; наявний попередній досвід розробки прототипу на Python, що дозволяє значно скоротити час розробки.

Порівняльні характеристики розглянутих мов програмування наведено в табл. 1.2.

Таблиця 1.2 – Порівняння мов програмування для реалізації проєкту

Мова	**Підтримка MediaPipe**	**Швидкість розробки**	**Продуктивність**	**Рекомендоване застосування**
Python	Офіційна (першокласна)	Висока	Середня	Прототипи, ML, CV
C++	Офіційна (нативна)	Низька	Дуже висока	Системне ПЗ, вбудовані системи
JavaScript	Відсутня	Висока	Середня	Веб-застосунки
Java	Неофіційна (JNI)	Середня	Висока	Корпоративні системи

2.1.4. Вибір середовища програмування

Середовище розробки (IDE) суттєво впливає на продуктивність програміста, зручність налагодження коду та якість підтримки використовуваних бібліотек. Для Python-розробки існує кілька популярних середовищ, кожне з яких має свої переваги та недоліки.

Visual Studio Code є легким і гнучким редактором коду з великою кількістю розширень. Підтримка Python у VS Code реалізована через

розширення Pylance та Python Extension. Проте VS Code не є повноцінним IDE – деякі функції (наприклад, рефакторинг, глибокий аналіз коду) поступаються спеціалізованим рішенням. Для великих проєктів налаштування середовища може займати значний час.

Spyder – IDE, орієнтоване на наукові обчислення та аналіз даних. Воно має вбудований оглядач змінних та можливість виконання коду в консолі в стилі MATLAB. Проте Spyder менш підходить для розробки повноцінних застосунків з класовою архітектурою та поступається PyCharm за інструментами рефакторингу.

Jupyter Notebook є зручним інструментом для інтерактивного аналізу даних та прототипування, але не підходить для розробки десктопних застосунків зі складною архітектурою, оскільки не призначений для роботи з багатофайловими проєктами.

PyCharm Community Edition обрано як основне середовище розробки. Це спеціалізоване IDE для Python від компанії JetBrains, що надає: інтелектуальне автодоповнення коду з урахуванням типів бібліотек MediaPipe, OpenCV та інших; вбудований відладчик з точками зупинки та інспекцією змінних; підтримку рефакторингу коду та автоматичного виявлення помилок; зручну інтеграцію з системою контролю версій Git; безкоштовну Community-версію з повним набором інструментів для локальної розробки. Порівняльні характеристики середовищ розробки наведено в табл. 1.3.

Таблиця 1.3 – Порівняння середовищ розробки для Python

IDE/ Редактор	**Переваги**	**Недоліки**	**Рекомендоване застосування**
Visual Studio Code	Легкий, гнучкий, розширюваний	Вимагає ручного налаштування	Фронтенд, легкі проєкти
Spyder	Оглядач змінних, наукові обчислення	Слабкий рефакторинг	Data Science, аналіз даних

Jupyter Notebook	Інтерактивне прототипування	Не для застосунків	Дослідження, ML-експерименти
PyCharm Community	Глибокий аналіз, відладчик, рефакторинг	Вища ресурсоємність	Повноцінні Python-застосунки

Таким чином, PyCharm Community Edition є оптимальним вибором для реалізації проєкту завдяки повній підтримці Python, вбудованим інструментам аналізу коду та безкоштовному поширенню.

2.1.5. Вибір системи управління базами даних

Система управління базами даних (СУБД) необхідна для зберігання конфігурацій вправ, параметрів поз та результатів тренувань. Вибір СУБД для локального десктопного застосунку суттєво відрізняється від вибору для веб-систем, оскільки пріоритет надається простоті розгортання та відсутності необхідності у сервері.

PostgreSQL та MySQL є потужними реляційними СУБД, що широко використовуються у веб-розробці. Вони підтримують складні SQL-запити, транзакції та забезпечують масштабованість. Однак обидві СУБД вимагають встановлення та запуску окремого серверного процесу, що є надлишковим для локального застосунку. Додатково ускладнюється розгортання на машині кінцевого користувача.

MongoDB є документно-орієнтованою NoSQL СУБД, що добре підходить для зберігання JSON-подібних структур. Проте MongoDB також вимагає окремого сервера та є надлишково складною для обсягів даних даного проєкту.

Для даного проєкту було обрано JSON-структуру у якості бази даних з таких причин: не потребує окремого серверного процесу – база даних зберігається в одному файлі на диску; вбудована підтримка в стандартній бібліотеці Python без встановлення додаткових залежностей; повна

підтримка SQL-синтаксису, включаючи транзакції та зовнішні ключі; висока надійність та широке застосування у вбудованих системах та десктопних застосунках; зручне перенесення бази даних разом з застосунком у вигляді єдиного файлу. JSON повністю забезпечує потреби проєкту в зберіганні структурованих даних про вправи та пози.

2.1.6. Вибір фреймворків та бібліотек

Правильний вибір бібліотек є критично важливим для ефективної реалізації функціоналу відстеження рухів та забезпечення якісного користувацького досвіду. Нижче наведено обґрунтування вибору кожної з ключових бібліотек проєкту.

MediaPipe [7] – бібліотека від Google для побудови пайплайнів обробки мультимедіа в реальному часі. Для даного проєкту використовується модуль Pose Landmarker, що детектує 33 ключові точки тіла людини (суглоби, кінцівки тощо) з координатами x, y, z та показником видимості. MediaPipe обрано через офіційну підтримку Python, оптимізацію для роботи в реальному часі (30+ FPS на CPU), високу точність розпізнавання та просту інтеграцію з OpenCV.

OpenCV [8] – найпоширеніша бібліотека комп'ютерного зору. У проєкті використовується для захоплення відеопотоку з веб-камери (cv2.VideoCapture), відображення вікна застосунку (cv2.imshow), малювання ліній скелету (mp_drawing.draw_landmarks) та обробки кадрів (зміна розміру, дзеркальне відображення, конвертація кольорів). OpenCV є стандартом де-факто для роботи з відеопотоком у Python.

Pillow (PIL) [11] – бібліотека обробки зображень. Використовується для відображення тексту українською мовою поверх відеокадру, оскільки стандартний метод cv2.putText не підтримує кирилицю. PIL Image та ImageDraw дозволяють накладати якісний антиальясований текст з підтримкою шрифтів TrueType.

Pygame [9] – бібліотека для розробки ігор та мультимедіа. У проєкті використовується виключно підсистема pygame.mixer для відтворення

звукових сповіщень (правильне виконання пози, завершення кроку). Pygame обрано через простоту API та надійну підтримку WAV-файлів без зовнішніх залежностей.

2.1.7. Вибір патернів проєктування

Патерни проєктування забезпечують структурованість коду, покращують його читабельність та спрощують подальше розширення системи. Для даного проєкту обрано набір патернів, що відповідають об'єктно-орієнтованій архітектурі та специфіці застосунку реального часу.

Патерн Стратегія (Strategy) застосовується для інкапсуляції алгоритмів перевірки правильності поз. Кожна поза йоги має свої унікальні умови правильного виконання (набір кутів суглобів, що перевіряються). Патерн Стратегія дозволяє визначити загальний інтерфейс перевірки пози та надати різні конкретні реалізації для кожної вправи, завантажуючи потрібну стратегію з JSON-файлу конфігурації динамічно під час виконання.

Патерн Команда (Command) реалізує прив'язку дій до кнопок інтерфейсу. Кожна кнопка (перемикання режиму, запис кроку, збереження JSON, вихід) інкапсулює виконувану дію у вигляді об'єкта. Це дозволяє легко додавати нові кнопки без зміни логіки обробки взаємодії, а також реалізувати буфер часу для запобігання випадковим натисканням.

Патерн Спостерігач (Observer) використовується для сповіщення системи про зміну стану пози. Коли MediaPipe виявляє нові координати ключових точок, модуль перевірки пози отримує сповіщення та обчислює кути суглобів. Такий підхід розділяє логіку захоплення відеопотоку та логіку оцінки пози.

Патерн Репозиторій (Repository) забезпечує абстрагування доступу до бази даних. Весь код роботи з JSON інкапсульовано у класі-репозиторії, що надає зручний API для завантаження та збереження конфігурацій вправ. Це дозволяє легко замінити JSON на інший механізм зберігання без зміни бізнес-логіки.

Патерн Одинак (Singleton) застосовується для класу YogaTrainerApp, що є центральним об'єктом, який управляє станом застосунку. Єдиний екземпляр цього класу зберігає поточний режим, індекс кроку, стан кнопок та останні координати ключових точок. Singleton гарантує узгодженість стану в усіх компонентах системи.

2.2 Проєктування застосунку підтримки виконання вправ гімнастики йога

2.2.1. Мета та задачі.

Метою розробки застосунку є створення програмного середовища для підтримки виконання вправ гімнастики йога, що надаватиме покрокові інструкції з виконання поз, розпізнаватиме тіло людини через веб-камеру, перевірятиме правильність виконання поз шляхом вимірювання кутів суглобів та надаватиме тренерам інструментарій для самостійного визначення нових вправ.

Система (надалі – застосунок) повинна забезпечувати комфортне та ефективне середовище для відпрацювання технік гімнастики йога без необхідності залучення живого тренера. Головний акцент робиться на автоматизованому контролі правильності виконання поз на основі точного вимірювання кутів між суглобами, а також на можливості тренерів розширювати бібліотеку вправ без написання програмного коду.

Застосунок повинен підтримувати два основних режими роботи. У режимі практики користувач отримує покрокові інструкції з виконання обраної вправи, а система в реальному часі відображає виміряні кути суглобів та порівнює їх з допустимими діапазонами. Після успішного утримання правильної пози впродовж заданого часу застосунок автоматично переходить до наступного кроку. У режимі тренера фахівець може виконати позу перед камерою, і система автоматично запише кути суглобів з відповідним допуском для подальшого використання у режимі практики.

2.2.2. Типи користувачів.

Застосунок підтримує два типи користувачів:

1. користувач-практик;
2. користувач-тренер.

Кожен тип має свої можливості:

1) Користувач-практик – особа, яка використовує застосунок для виконання вправ гімнастики йога. Його можливості включають: запуск застосунку та вибір вправи через аргумент командного рядка; виконання вправ у покроковому режимі з автоматичним переходом між кроками; перегляд поточних кутів суглобів та їх відповідності нормі; отримання звукових сповіщень при правильному виконанні пози; вмикання та вимикання відображення скелету відстеження; вихід із застосунку.

2) Користувач-тренер – фахівець з гімнастики йога, який формує бібліотеку вправ. Його можливості включають всі можливості практика, а також: переключення у режим тренера для запису нових поз; виконання пози перед камерою та автоматичний запис поточних кутів суглобів як еталонних; перехід між кроками вправи для послідовного запису всіх положень; збереження записаної конфігурації вправи у JSON-файл для подальшого використання.

2.2.3. Функціональні вимоги

Розглянемо функціональні вимоги (ФВ) до застосунку.

Прецедент 1 - Структурування опису йога-вправ

ФВ1.1 - користувач передає програмі неструктурований опис йога-вправ;

ФВ1.2 - програма структурує опис йога-вправ;

ФВ1.3 - програма передає структурований опис йога-вправи користувачу для узгодження;

ФВ1.4 - користувач за необхідністю виправляє структурований опис йога-вправи;

ФВ1.5 - користувач передає програмі виправлений структурований опис йога-вправи;

ФВ1.6 - програма зберігає структурований опис йога-вправи у базі даних

Прецедент 2 - Підготовка до йога-вправи

ФВ2.1 - програма надає користувачу список йога-вправ;

ФВ2.2 - користувач обирає зі списку йога-вправу;

ФВ2.3 - програма вказує місце розташування вебкамери відносно користувача;

ФВ2.4 - програма передає користувачу опис обраної йога-вправи;

Прецедент 3 - Виконанння йога-вправ

ФВ2.1 - користувач отримує від програми опис йога-вправи;

ФВ2.2 - програма отримує положення тіла користувача;

ФВ2.3 - програма коментує правильне/неправильне положення тіла користувача із урахуванням опису йога-вправи;

ФВ2.4 - програма надає рекомендації користувачу щодо напрямку зміни положення тіла користувача;

ФВ2.5 - програма повідомляє користувача про успішність виконання йога-вправи;

2.2.4. Нефункціональні вимоги

Розглянемо нефункціональні вимоги (НФВ) до застосунку.

НФВ1 Продуктивність.

НФВ1.1 Застосунок повинен обробляти відеопотік зі швидкістю не менше 15 кадрів на секунду на ПК з процесором Intel Core i5 або аналогічним.

НФВ1.2 Затримка між фізичним рухом користувача та відображенням оновленого кута суглобу не повинна перевищувати 100 мс.

НФВ1.3 Час завантаження конфігурації вправи з JSON-файлу не повинен перевищувати 500 мс.

НФВ2 Зручність використання.

НФВ2.1 Інтерфейс повинен відображати всю необхідну інформацію (поточний крок, кути суглобів, кнопки) в межах одного вікна без прокрутки.

НФВ2.2 Кнопки повинні мати достатній розмір (200×50 пікселів) для надійного розпізнавання при наведенні пальця.

НФВ2.3 Текстові повідомлення повинні відображатися кирилицею з достатнім розміром шрифту (не менше 14pt) для читабельності.

НФВ3 Надійність.

НФВ3.1 Застосунок повинен коректно обробляти ситуації, коли тіло людини не виявлено в кадрі, без аварійного завершення.

НФВ3.2 Помилки завантаження файлів зображень або звуків повинні оброблятися з виведенням повідомлення без зупинки роботи.

НФВ3.3 При відключенні веб-камери застосунок повинен виводити повідомлення про помилку та завершити роботу коректно.

НФВ4 Портативність.

НФВ4.1 Застосунок повинен коректно працювати на операційних системах Windows 10/11 та Linux (Ubuntu 20.04+).

НФВ4.2 Усі залежності повинні бути встановлювані через pip без необхідності компіляції нативних модулів.

НФВ5 Обслуговування та підтримка.

НФВ5.1 Конфігурації вправ повинні зберігатися у людиночитабельному форматі JSON, доступному для ручного редагування.

НФВ5.2 Код застосунку повинен містити коментарі для всіх основних функцій та класів.

2.2.5. Ідентифікація архетипу

Розроблюваний застосунок належить до архетипу Desktop Application (десктопний застосунок), оскільки він є локальним програмним

забезпеченням, що встановлюється та виконується безпосередньо на комп'ютері користувача без потреби у мережевому з'єднанні.

Обґрунтування вибору архетипу:

3. локальне виконання: застосунок працює повністю на локальному ПК, обробляє відеопотік у реальному часі без передачі даних на зовнішні сервери, що забезпечує низьку затримку та захист персональних відеоданих;
4. пряма робота з апаратним забезпеченням: застосунок безпосередньо звертається до веб-камери через OpenCV, що потребує прямого доступу до пристроїв введення – характерна риса десктопних застосунків;
5. офлайн-функціонування: всі обчислення (нейромережева обробка MediaPipe, тригонометричні розрахунки кутів) виконуються локально без Інтернету;
6. файлова система: конфігурації вправ зберігаються у JSON-файлах на локальному диску, що є типовим підходом для десктопних застосунків;
7. нативне вікно: відображення відеопотоку реалізовано через нативне вікно OpenCV, а не через браузер чи веб-технології.

Таким чином, вибір архетипу Desktop Application є єдиним доцільним рішенням для застосунку, що вимагає прямого доступу до веб-камери з мінімальною затримкою та повноцінної офлайн-роботи.

2.2.6. Уявлення інтерфейсу користувача

Інтерфейс застосунку розроблено з урахуванням принципів зручності використання в умовах активних фізичних вправ, коли користувач не може використовувати мишу чи клавіатуру. Всі елементи управління виконано у вигляді екранних кнопок, що активуються наведенням пальця.

Головне вікно застосунку з роздільністю 1280×960 пікселів розділено на три функціональні зони. Основна зона займає більшу частину вікна та відображає відеопотік з веб-камери в дзеркальному режимі. Поверх

відеопотоку накладається скелет відстеження MediaPipe (зелені точки суглобів та червоні лінії з'єднань) та інформаційна панель у лівій частині.

Інформаційна панель розташована у верхній лівій частині вікна та відображає: поточний режим роботи (Практика або Тренер); номер та опис поточного кроку вправи; назву частини тіла, задіяної у вправі; список кутів суглобів з поточними виміряними значеннями, допустимими діапазонами та індикаторами правильності (✓/✗); зворотний відлік часу утримання правильної пози.

Панель кнопок розташована в нижній частині вікна та містить шість кнопок: Mode Switch (перемикання режиму), Toggle Skeleton (скелет), Record Step (запис кроку), Next Step (наступний крок), Save JSON (збереження) та Exit (вихід). Кожна кнопка має розмір 200×50 пікселів та змінює колір (синій → зелений) при наведенні пальця.

2.2.7. Логічне уявлення

Логічна архітектура застосунку побудована на основі центрального класу YogaTrainerApp, що реалізує патерн Одинак та управляє всіма компонентами системи. Клас інкапсулює поточний стан (режим, крок, таймери) та координує взаємодію між модулями захоплення відео, розпізнавання поз та відображення інтерфейсу.

Модуль захоплення відеопотоку (VideoCapture) відповідає за ініціалізацію веб-камери через OpenCV та надання сирих кадрів для подальшої обробки. Модуль аналізу поз (PoseAnalyzer) реалізує взаємодію з MediaPipe Pose та обчислення кутів між суглобами через функцію calculate_angle(). Модуль перевірки вправ (ExerciseChecker) завантажує конфігурацію з JSON та реалізує патерн Стратегія для перевірки правильності виконання.

Модуль відображення (Renderer) відповідає за конвертацію кадрів між форматами OpenCV та PIL, відображення тексту українською мовою та малювання кнопок. Модуль взаємодії (InteractionHandler) реалізує патерн Команда та відстежує стан кнопок на основі координат пальців. Модуль

збереження даних (DataRepository) реалізує патерн Репозиторій та забезпечує завантаження/збереження конфігурацій у JSON.

2.2.8. Уявлення розгортання

Застосунок розгортається як локальна Python-програма на комп'ютері кінцевого користувача. Процес розгортання включає встановлення Python 3.8 або вище, встановлення залежностей через pip, підключення веб-камери та запуск застосунку з вказівкою конфігураційного JSON-файлу.

Для роботи застосунку необхідні такі апаратні компоненти: ПК або ноутбук з процесором Intel Core i5 або аналогічним (для обробки MediaPipe в реальному часі); веб-камера з роздільністю не менше 640×480 при 30 FPS; оперативна пам'ять об'ємом не менше 4 ГБ; аудіовихід для відтворення звукових сповіщень.

2.2.9. Уявлення шарів

Архітектура застосунку реалізована як чотирирівнева система, де кожен шар відповідає за чітко визначену функціональність та взаємодіє лише з суміжними шарами.

Шар представлення (Presentation Layer) включає модуль Renderer, що відповідає за відображення відеопотоку, накладання тексту та кнопок, та модуль InteractionHandler, що обробляє жести користувача. Цей шар перетворює внутрішній стан застосунку на зрозумілий візуальний інтерфейс.

Прикладний шар (Application Layer) представлений класом YogaTrainerApp, що координує роботу всіх компонентів, управляє станом (режим, поточний крок, таймери кнопок) та реалізує основний цикл обробки відеопотоку.

Шар бізнес-логіки (Domain Layer) включає модуль PoseAnalyzer для обчислення кутів суглобів та ExerciseChecker для перевірки правильності виконання вправ. Саме тут реалізовані основні алгоритми оцінки поз та логіка переходу між кроками.

Інфраструктурний шар (Infrastructure Layer) забезпечує взаємодію з апаратним забезпеченням (VideoCapture через OpenCV), нейромережевою бібліотекою (MediaPipe), та файловою системою (JSON-файли конфігурацій).

2.2.10. Уявлення процесів

Основним процесом застосунку є головний цикл обробки відеопотоку, що виконується у нескінченному циклі while cap.isOpened(). Цикл складається з послідовних етапів: захоплення кадру, обробка MediaPipe, оновлення стану та відображення результату.

На першому етапі OpenCV захоплює новий кадр з веб-камери та виконує його попередню обробку: горизонтальне відзеркалення та масштабування до 1280×960. Потім кадр конвертується з BGR у RGB для MediaPipe.

На другому етапі MediaPipe Pose обробляє RGB-кадр та повертає координати 33 ключових точок тіла. Якщо ключові точки виявлені, кадр конвертується у PIL Image для подальшого відображення тексту кирилицею.

На третьому етапі InteractionHandler перевіряє, чи знаходиться вказівний палець користувача в межах будь-якої кнопки. При виявленні наведення запускається або оновлюється таймер кнопки. Після спрацювання таймера виконується відповідна команда.

На четвертому етапі залежно від режиму (практика або тренер) виконується відповідна логіка: обчислення кутів суглобів, перевірка правильності пози, відлік часу утримання або відображення поточних значень кутів для запису.

2.2.11. Уявлення даних

Основною структурою даних застосунку є JSON-файл конфігурації вправи. Файл має кореневий об'єкт YogaPose, що містить назву вправи та масив кроків (steps). Кожен крок описується полями: номер кроку (stepNumber), текстовий опис (description), назва задіяної частини тіла

(bodyPart), тривалість утримання у секундах (duration) та масив кутів (angles).

Кожен елемент масиву angles описує один вимірюваний кут та містить три орієнтири (joint1, base_joint, joint2 – рядкові ідентифікатори з простору імен MediaPipe PoseLandmark), а також допустимий діапазон (min_angle, max_angle у градусах). Такий формат дозволяє описати довільну позу без програмування – достатньо знати назви суглобів та допустимі кути між ними.

2.2.12. Уявлення комунікацій

Комунікація між компонентами застосунку здійснюється виключно через виклики методів всередині одного Python-процесу, що є типовим для монолітних десктопних застосунків. Зовнішні мережеві комунікації відсутні.

Основний потік даних починається з VideoCapture, що передає сирі BGR-кадри до головного циклу. Кадри конвертуються та передаються до MediaPipe, який повертає landmarks у вигляді об'єктів з координатами та показниками видимості. PoseAnalyzer обчислює кути між суглобами та повертає словник значень. ExerciseChecker порівнює виміряні кути з конфігурацією з JSON та повертає булеве значення правильності та часткові результати для кожного кута.

InteractionHandler отримує координати пальців з landmarks та стан кнопок, повертаючи список спрацьованих команд. Renderer отримує кадр, результати перевірки, стан кнопок та текстові повідомлення, формуючи фінальний кадр для відображення. DataRepository обмінюється даними з файловою системою (JSON) через стандартний Python API. На рис. 2.8 наведено діаграму комунікацій.

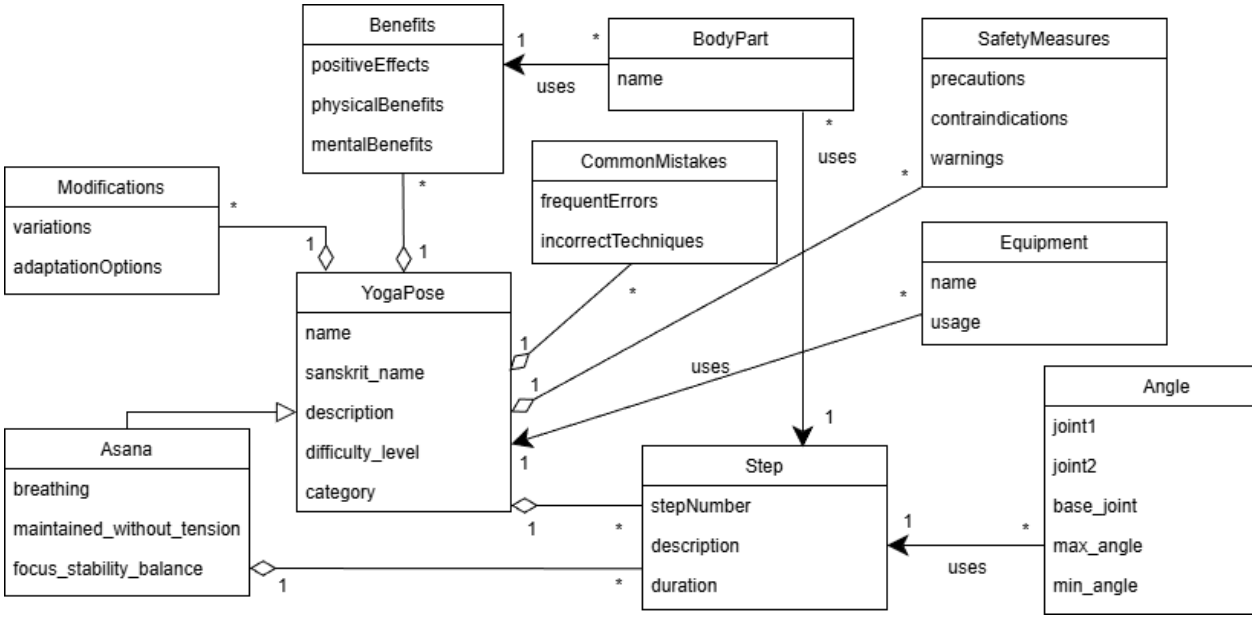

Рисунок 2.8 – Діаграма комунікацій застосунку

2.2.13. Опис стеку технологій

У даному підрозділі наведено перелік усіх технологій, бібліотек та засобів розробки, що використовуються у проєкті:

1) Мови програмування:

а) Python 3.10 – основна мова програмування застосунку;

2) Бібліотеки комп'ютерного зору та обробки зображень:

а) MediaPipe 0.10 – розпізнавання ключових точок тіла (Pose Landmarker);

б) OpenCV (cv2) 4.8 – захоплення відеопотоку, відображення вікна, малювання скелету;

в) Pillow (PIL) 10.0 – відображення тексту кирилицею поверх відеокадру;

г) NumPy 1.24 – матричні операції та конвертація зображень;

3) Мультимедіа:

а) Pygame 2.5 – відтворення звукових сповіщень (модуль mixer);

4) Зберігання даних:

а) JSON (модуль json) – зберігання та завантаження конфігурацій вправ;

5) Інструменти розробки:

а) PyCharm Community Edition 2024 – основне середовище розробки;

б) Git – система контролю версій;

в) GitHub – зберігання репозиторію проєкту;

Даний стек технологій забезпечує ефективну обробку відеопотоку в реальному часі, зручний механізм визначення нових вправ через JSON без програмування та повноцінну офлайн-роботу застосунку.

2.3 Результат роботи

У результаті роботи була створена повнофункціональна програма-тренер з поз йоги на Python. Програма запускається командою:

python main.py warrior2.json

де warrior2.json — файл конфігурації вправи. Якщо аргумент не вказано, програма використовує файл warrior2.json за замовчуванням.

Повний код програми буде наведено у Додатку 1.

Програма успішно реалізує всі поставлені вимоги. У режимі практики користувач бачить поточні кути своїх суглобів у реальному часі, отримує візуальний зворотний зв'язок (зелений колір - правильно, червоний - неправильно) та звуковий сигнал при успішному виконанні. У режимі тренера фахівець може зафіксувати нову позу та зберегти її для подальшого використання.

Використання JSON-формату для конфігурацій вправ робить програму легко розширюваною – для додавання нової вправи не потрібно змінювати код, достатньо створити відповідний JSON-файл.

Нижче зображено скріншоти програми під час роботи.

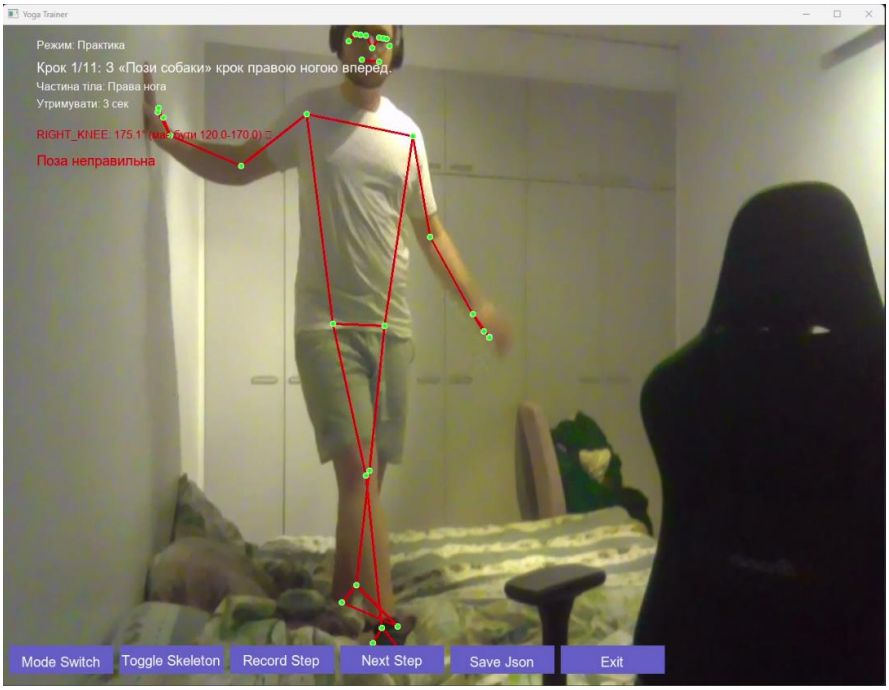

Рис. 1 – Скріншот програми під час роботи, користувач не зайняв правильну позу

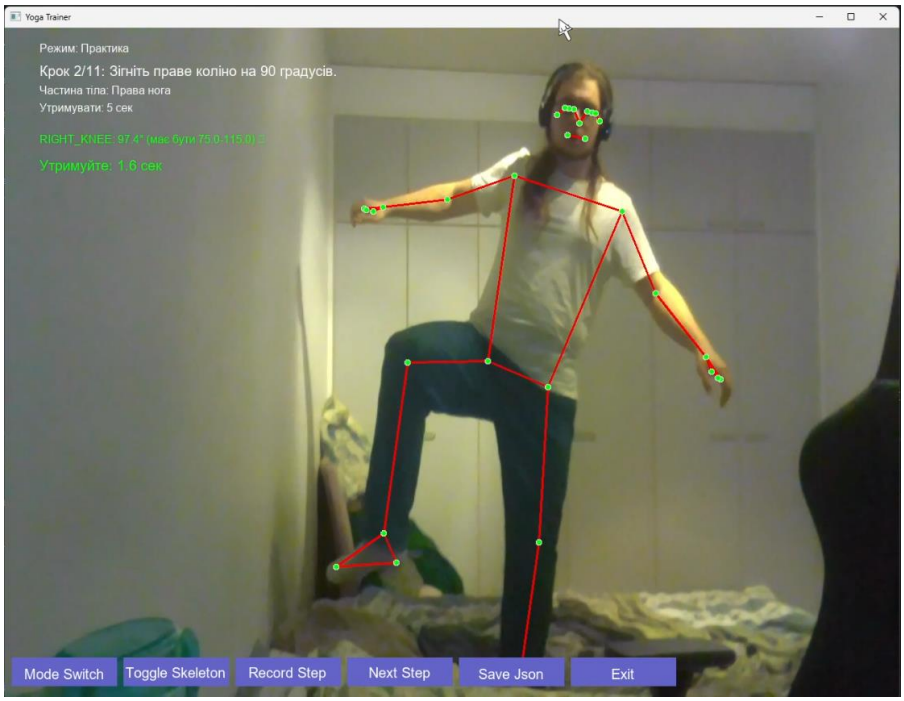

Рис. 2 – Скріншот програми під час роботи, користувач зайняв позу для виконання кроку 2 асани «Воїн II»

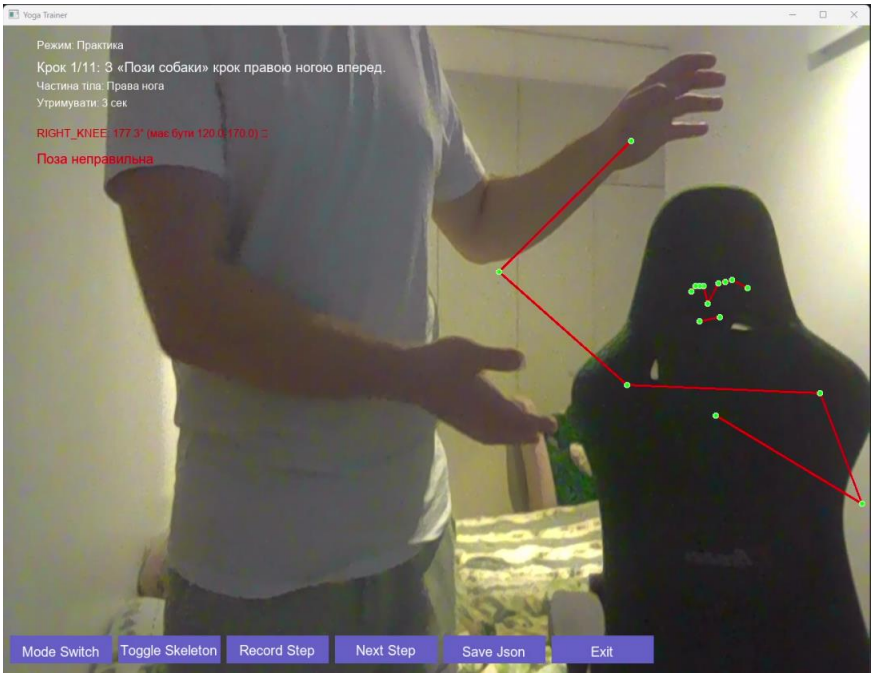

Рис. 3 – Скріншот програми під час роботи, Mediapipe розпізнав сторонній об’єкт як користувача. При зміні положення стороннього об’єкту програма коректно розпізнала користувача та продовжила нормальні операції.

Висновки до розділу 2

У розділі 2.1 проведено огляд існуючих аналогів застосунків для йоги та фітнесу з елементами штучного інтелекту, що дозволило визначити їхні сильні та слабкі сторони. Встановлено, що жоден з розглянутих аналогів не поєднує в собі відстеження поз через камеру з можливістю тренеру самостійно визначати та зберігати нові вправи без програмування.

Обґрунтовано вибір технологічного стеку: Python обраний як основна мова програмування завдяки першокласній підтримці MediaPipe та OpenCV, а також багатій екосистемі бібліотек для комп'ютерного зору. Монолітна архітектура визнана оптимальною для локального десктопного застосунку. PyCharm Community Edition забезпечує повноцінне середовище розробки безкоштовно. JSON обрано як структуру даних завдяки вбудованій підтримці в Python та відсутності потреби у сервері.

Визначено набір бібліотек: MediaPipe для розпізнавання ключових точок тіла, OpenCV для роботи з відеопотоком, Pillow для відображення

кирилиці, Pygame для звукових сповіщень та NumPy для матричних обчислень. Обрано патерни проєктування Strategy, Command, Observer, Repository та Singleton, що забезпечують чисту, розширювану архітектуру. Сукупність прийнятих рішень формує надійний технологічний фундамент для реалізації поставлених задач.

У розділі 2.2 проведено повне проєктування застосунку підтримки виконання вправ гімнастики йога. Визначено мету, задачі та два типи користувачів: практик та тренер, кожен з яких має свій набір можливостей.

Сформульовано функціональні вимоги, що охоплюють захоплення відеопотоку, режим практики з покроковим виконанням та автоматичною перевіркою поз, режим тренера для запису нових вправ та взаємодію через жести. Визначено нефункціональні вимоги щодо продуктивності (15+ FPS, затримка до 100 мс), зручності використання, надійності та портативності.

Застосунок ідентифіковано як десктопний застосунок (Desktop Application). Розроблено концепцію інтерфейсу з трьома функціональними зонами та системою шести кнопок для безконтактного управління. Логічна архітектура реалізована через чотири шари (представлення, прикладний, бізнес-логіки, інфраструктурний), що забезпечує чисте розділення відповідальностей.

Описано структуру JSON-файлів конфігурації вправ. Уявлення комунікацій підтвердило відсутність зовнішніх мережевих залежностей – всі компоненти взаємодіють виключно через виклики методів всередині одного Python-процесу. Визначено повний стек технологій, що забезпечує ефективну реалізацію всіх поставлених функціональних вимог.

У розділі 2.3 протестовано та успішно випробовано результуючу програму.

ВИСНОВКИ

Під час цієї практики було прослухано лекцію від фахівців ТОВ «IT-КАРКАС», де ознайомився. Знання мають практичне значення для майбутньої роботи у галузі.

Також було виконано індивідуальне завдання та розробилено вдосконалену програму-тренер з поз йоги. У порівнянні з попередньою версією, нова програма використовує точне вимірювання кутів суглобів замість простої перевірки відносного розташування, підтримує два режими роботи, JSON-конфігурацію вправ та рендеринг тексту через PIL для коректного відображення кирилиці. Програма побудована на основі класу, що відповідає принципу єдиної відповідальності та спрощує її подальшу підтримку і розширення.

Програма може бути розвинута за допомогою інтеграції значеного покращення UI/UX, що буде частиною моєї дипломної роботи, розширення бази вправ, а також допрацювання режиму тренера для створення нових вправ повністю у програмі.

Результати робіт із передипломної практики представлено на конференції [6] у вигляді тез, та було додано до збірнику з цієї конференції.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Воловик Н. І. (2024). *Основи фітнес-йоги: Навчальний посібник для студентів закладів вищої освіти*. Видавництво Український державний університет імені Михайла Драгоманова.
2. Sahare, Prajwal. (2023). Gym & Yoga Trainer Using Machine Learning. *International Journal for Research in Applied Science and Engineering Technology.* 11. 1461-1465. https://doi.org/10.22214/ijraset.2023.53820
3. Безугла А. І., & Блажко О. А. (2025). Особливості структури бази даних JSON-формату опису фізичних вправ для їх комп'ютерної ігровізації. У *Технології та суспільство: взаємодія, вплив, трансформація: Матеріали V Міжнародної наукової конференції* (pp. 213–217). УКРЛОГОС Груп. https://doi.org/10.5281/zenodo.18107960
4. Google AI. (2025). *Pose landmark detection guide* (MediaPipe Pose Landmarker). https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker
5. Attouche L., Baazizi M.-A., et al. (2024). Validation of Modern JSON Schema: Formalization and Complexity. *Proceedings of the ACM on Programming Languages, Volume 8, Issue POPL.* 49, 1451-1481. https://doi.org/10.1145/3632891
6. Сучков О. І., Блажко О. А. Особливості структури бази даних опису вправ гімнастики йога для розробки віртуального тренера *Інтелектуальний ресурс сьогодення: наукові задачі, розвиток та запитання: збірник наукових праць з матеріалами VI Міжнародної наукової конференції, м. Черкаси, 27 лютого, 2026 р. / Міжнародний центр наукових досліджень. – Вінниця: ТОВ «УКРЛОГОС Груп, 2026.* С. 205-209. DOI: https://doi.org/10.5281/zenodo.18811695
7. MediaPipe Solutions guide. URL: https://ai.google.dev/edge/mediapipe/solutions/guide (дата звернення - 10.02.2026)

8. OpenCV Documentation. URL: https://docs.opencv.org (дата звернення - 19.02.2026)
9. Pygame Documentation. URL: https://www.pygame.org/docs (дата звернення - 19.02.2026)
10. Python Documentation. URL: https://docs.python.org/3 (дата звернення - 17.02.2026)
11. Pillow (PIL) Documentation. URL: https://pillow.readthedocs.io (дата звернення - 21.02.2026)
12. DeepSeek. URL: https://chat.deepseek.com (дата звернення - 26.02.2026)

ДОДАТОК 1 – ПОВНИЙ ПРОГРАМНИЙ КОД YogaTrainerApp

```
import cv2
import mediapipe as mp
mp_pose = mp.solutions.pose
import numpy as np
import json
import time
import math
import sys
import pygame
from PIL import Image, ImageDraw, ImageFont

# ==================== Ініціалізація MediaPipe ====================
mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
pose = mp_pose.Pose(min_detection_confidence=0.5, min_tracking_confidence=0.5)

# ==================== Ініціалізація звуку ====================
pygame.mixer.init()
SOUND_CORRECT = "correct.wav"           # коли поза стала правильною
SOUND_STEP_DONE = "step_complete.wav"   # коли крок витримано
try:
    sound_correct = pygame.mixer.Sound(SOUND_CORRECT)
except:
    sound_correct = None
try:
    sound_step_done = pygame.mixer.Sound(SOUND_STEP_DONE)
except:
    sound_step_done = None

# ==================== Константи ====================
WINDOW_WIDTH = 1280
WINDOW_HEIGHT = 960
BUTTON_WIDTH = 200
BUTTON_HEIGHT = 50
MARGIN_DEG = 15.0          # запас для режиму тренера

# Кольори
COLOR_BG = (50, 50, 50)
COLOR_BUTTON = (100, 100, 200)
COLOR_BUTTON_HOVER = (0, 200, 0)
COLOR_TEXT = (255, 255, 255)
COLOR_CORRECT = (0, 255, 0)
COLOR_INCORRECT = (255, 0, 0)

# Шрифт (PIL)
try:
    font = ImageFont.truetype("C:/Windows/Fonts/arial.ttf", 20)
    font_small = ImageFont.truetype("C:/Windows/Fonts/arial.ttf", 16)
except:
```

```
    font = ImageFont.load_default()
    font_small = ImageFont.load_default()

# ==================== Допоміжні функції ====================
def calculate_angle(landmark1, landmark2, landmark3):
    """
    Обчислює кут між трьома орієнтирами (вершина — landmark2).
    Повертає кут у градусах (0-180).
    """
    x1, y1 = landmark1.x, landmark1.y
    x2, y2 = landmark2.x, landmark2.y
    x3, y3 = landmark3.x, landmark3.y

    v1 = (x1 - x2, y1 - y2)
    v2 = (x3 - x2, y3 - y2)

    dot = v1[0]*v2[0] + v1[1]*v2[1]
    mag1 = math.hypot(v1[0], v1[1])
    mag2 = math.hypot(v2[0], v2[1])

    if mag1 * mag2 == 0:
        return 0.0

    cos_angle = dot / (mag1 * mag2)
    angle = math.degrees(math.acos(max(-1.0, min(1.0, cos_angle))))
    return angle

def load_pose_from_json(filepath):
    """Завантажує вправу з JSON-файлу."""
    with open(filepath, 'r', encoding='utf-8') as f:
        data = json.load(f)
    return data["YogaPose"]

def save_pose_to_json(filepath, pose_data):
    """Зберігає вправу у JSON-файл (з обгорткою YogaPose)."""
    with open(filepath, 'w', encoding='utf-8') as f:
        json.dump({"YogaPose": pose_data}, f, ensure_ascii=False, indent=2)

def get_landmark_index(name):
    """Повертає індекс landmark за його рядковим ім'ям (наприклад, 'RIGHT_HIP')."""
    try:
        return mp_pose.PoseLandmark[name].value
    except:
        print(f"Помилка: невідомий landmark {name}")
        return None

def get_current_angles(landmarks, angle_specs):
    """
    Для списку специфікацій кутів (кожна з joint1, base_joint, joint2)
    повертає список обчислених кутів.
    """
    angles = []
```

```
    for spec in angle_specs:
        idx1 = get_landmark_index(spec["joint1"])
        idx2 = get_landmark_index(spec["base_joint"])
        idx3 = get_landmark_index(spec["joint2"])
        if None in (idx1, idx2, idx3):
            angles.append(None)
            continue
        angle = calculate_angle(landmarks[idx1], landmarks[idx2], landmarks[idx3])
        angles.append(angle)
    return angles

def check_angles_within_range(angles, angle_specs):
    """Перевіряє, чи всі кути знаходяться в заданих діапазонах."""
    if len(angles) != len(angle_specs):
        return False
    for i, spec in enumerate(angle_specs):
        if angles[i] is None:
            return False
        if angles[i] < spec["min_angle"] or angles[i] > spec["max_angle"]:
            return False
    return True

def draw_text_pil(frame_pil, text, pos, color=COLOR_TEXT, font=font):
    """Малює текст на PIL-зображенні."""
    draw = ImageDraw.Draw(frame_pil)
    draw.text(pos, text, fill=color, font=font)

def draw_button(draw, rect, text, hover=False):
    """Малює кнопку з текстом."""
    color = COLOR_BUTTON_HOVER if hover else COLOR_BUTTON
    draw.rectangle(rect, fill=color)
    # Центруємо текст
    bbox = draw.textbbox((0, 0), text, font=font)
    text_w = bbox[2] - bbox[0]
    text_h = bbox[3] - bbox[1]
    x = rect[0] + (rect[2] - rect[0] - text_w) // 2
    y = rect[1] + (rect[3] - rect[1] - text_h) // 2
    draw.text((x, y), text, fill=COLOR_TEXT, font=font)

# ==================== Основний клас програми ====================
class YogaTrainerApp:
    def __init__(self, json_path):
        self.json_path = json_path
        self.pose_data = load_pose_from_json(json_path)
        self.steps = self.pose_data["steps"]
        self.total_steps = len(self.steps)

        # Стан програми
        self.mode = "practice"          # "practice" або "trainer"
        self.current_step = 0
        self.hold_start_time = None     # час, коли поза стала правильною
        self.step_completed = False
```

```
        self.show_skeleton = True
        self.recording = False          # чи йде запис у режимі тренера

        # Таймери для кнопок (запобігання випадковим натисканням)
        self.button_press_start = {}

        # Для збереження записаних кутів у режимі тренера
        self.recorded_angles = []        # не використовується, записуємо відразу в steps

        # Визначаємо кнопки: назва -> (x, y, ширина, висота)
        self.buttons = {
            "mode_switch": (10, WINDOW_HEIGHT - 60, 150, 40),
            "toggle_skeleton": (170, WINDOW_HEIGHT - 60, 150, 40),
            "record_step": (330, WINDOW_HEIGHT - 60, 150, 40),
            "next_step": (490, WINDOW_HEIGHT - 60, 150, 40),
            "save_json": (650, WINDOW_HEIGHT - 60, 150, 40),
            "exit": (810, WINDOW_HEIGHT - 60, 150, 40),
        }

        # Для звуків
        self.last_sound_time = 0
        self.sound_cooldown = 2.0

    def run(self):
        cap = cv2.VideoCapture(0)  # спробуйте 0 або 1
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, WINDOW_WIDTH)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, WINDOW_HEIGHT)

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            # Дзеркальне відображення
            frame = cv2.flip(frame, 1)
            frame = cv2.resize(frame, (WINDOW_WIDTH, WINDOW_HEIGHT))
            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            result = pose.process(rgb)

            # Створюємо копію для малювання
            frame_pil = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
            draw = ImageDraw.Draw(frame_pil)

            # Отримуємо landmarks якщо є
            landmarks = None
            if result.pose_landmarks:
                landmarks = result.pose_landmarks.landmark
                self.last_landmarks = landmarks
                if self.show_skeleton:
                    # Малюємо скелет на оригінальному фреймі (BGR)
                    mp_drawing.draw_landmarks(
                        frame, result.pose_landmarks, mp_pose.POSE_CONNECTIONS,
```

```
                        mp_drawing.DrawingSpec(color=(0,255,0), thickness=2),
                        mp_drawing.DrawingSpec(color=(0,0,255), thickness=2)
                    )
                    # Оновлюємо frame_pil (скелет вже намальовано на frame, але ми використовуємо
frame_pil для тексту)
                    # Тому краще малювати скелет безпосередньо на frame_pil через малювання ліній,
але для простоти залишимо так:
                    # Скелет вже є на frame, а ми конвертуємо frame в frame_pil пізніше. Тому
треба конвертувати frame зі скелетом.
                    # Тому перетворимо frame (зі скелетом) назад у PIL:
                    frame_pil = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
                    draw = ImageDraw.Draw(frame_pil)

            # Обробка натискань пальцем
            if landmarks:
                self.handle_finger_interaction(landmarks, frame_pil)

            # Відображення інформації залежно від режиму
            if self.mode == "practice":
                self.draw_practice_mode(frame_pil, draw, landmarks)
            else:  # trainer
                self.draw_trainer_mode(frame_pil, draw, landmarks)

            # Малюємо кнопки
            self.draw_buttons(draw)

            # Конвертуємо назад у OpenCV для показу
            frame = cv2.cvtColor(np.array(frame_pil), cv2.COLOR_RGB2BGR)
            cv2.imshow('Yoga Trainer', frame)

            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

        cap.release()
        cv2.destroyAllWindows()

    def handle_finger_interaction(self, landmarks, frame_pil):
        h, w = frame_pil.size[1], frame_pil.size[0]
        finger_points = [
            landmarks[mp_pose.PoseLandmark.LEFT_INDEX],
            landmarks[mp_pose.PoseLandmark.RIGHT_INDEX]
        ]
        current_time = time.time()
        hovered_buttons = set()

        for finger in finger_points:
            if finger.visibility < 0.5:
                continue
            fx, fy = int(finger.x * w), int(finger.y * h)

            for btn_name, (bx, by, bw, bh) in self.buttons.items():
                if bx <= fx <= bx + bw and by <= fy <= by + bh:
```

```
                    hovered_buttons.add(btn_name)
                    # Якщо таймер не запущено або скинуто — запускаємо
                    if (btn_name not in self.button_press_start or
                            self.button_press_start[btn_name] is None):
                        self.button_press_start[btn_name] = current_time
                    else:
                        # Якщо таймер вже є і минуло більше 1 секунди — активуємо
                        elapsed = current_time - self.button_press_start[btn_name]
                        if elapsed > 1.0:
                            self.execute_button_action(btn_name)
                            self.button_press_start[btn_name] = None  # скидаємо після активації
                    break  # виходимо з циклу кнопок для цього пальця (одночасно тільки одна
кнопка)

        # Скидаємо таймери для кнопок, на які не наведено жоден палець
        for btn_name in list(self.button_press_start.keys()):
            if btn_name not in hovered_buttons:
                self.button_press_start[btn_name] = None

    def execute_button_action(self, btn_name):
        """Виконує дію при натисканні кнопки."""
        if btn_name == "mode_switch":
            self.mode = "trainer" if self.mode == "practice" else "practice"
            self.current_step = 0
            self.hold_start_time = None
            print(f"Режим змінено на {self.mode}")
        elif btn_name == "toggle_skeleton":
            self.show_skeleton = not self.show_skeleton
        elif btn_name == "record_step":
            if self.mode == "trainer" and self.current_step < self.total_steps:
                self.record_current_step()
        elif btn_name == "next_step":
            if self.mode == "trainer" and self.current_step < self.total_steps - 1:
                self.current_step += 1
                self.hold_start_time = None
        elif btn_name == "save_json":
            # Зберігаємо поточні дані в новий файл
            new_path = self.json_path.replace(".json", "_trained.json")
            save_pose_to_json(new_path, self.pose_data)
            print(f"Збережено у {new_path}")
        elif btn_name == "exit":
            cv2.destroyAllWindows()
            sys.exit()

    def draw_buttons(self, draw):
        """Малює всі кнопки з підсвіткою, якщо наведено палець."""
        for btn_name, (bx, by, bw, bh) in self.buttons.items():
            hover = (btn_name in self.button_press_start and
                     self.button_press_start[btn_name] is not None)
            draw_button(draw, (bx, by, bx+bw, by+bh), btn_name.replace("_", " ").title(), hover)

    def draw_practice_mode(self, frame_pil, draw, landmarks):
```

```
        """Відображає інформацію в режимі практики."""
        if self.current_step >= self.total_steps:
            draw_text_pil(frame_pil, "Вправа завершена!", (50, 50), COLOR_CORRECT, font)
            return

        step = self.steps[self.current_step]
        step_num = step["stepNumber"]
        desc = step["description"]
        duration = step["duration"]
        body_part = step["bodyPart"]
        angle_specs = step["angles"]

        # Виводимо заголовок
        draw_text_pil(frame_pil, f"Режим: Практика", (50, 20), COLOR_TEXT, font_small)
        draw_text_pil(frame_pil,  f"Крок  {step_num}/{self.total_steps}:  {desc}",  (50,  50),
COLOR_TEXT, font)
        draw_text_pil(frame_pil, f"Частина тіла: {body_part}", (50, 80), COLOR_TEXT, font_small)
        draw_text_pil(frame_pil, f"Утримувати: {duration} сек", (50, 105), COLOR_TEXT, font_small)

        if landmarks is None:
            draw_text_pil(frame_pil, "Тіло не видно", (50, 150), COLOR_INCORRECT, font)
            return

        # Обчислюємо поточні кути
        angles = get_current_angles(landmarks, angle_specs)
        correct = check_angles_within_range(angles, angle_specs)

        # Відображаємо кути
        y_offset = 150
        for i, spec in enumerate(angle_specs):
            if angles[i] is not None:
                status = "✓" if (angles[i] ≥ spec["min_angle"] and angles[i] ≤ spec["max_angle"])
else "✗"
                color = COLOR_CORRECT if status == "✓" else COLOR_INCORRECT
                text = f"{spec['base_joint']}: {angles[i]:.1f}° (має бути {spec['min_angle']}-
{spec['max_angle']}) {status}"
            else:
                color = COLOR_INCORRECT
                text = f"{spec['base_joint']}: недоступно"
            draw_text_pil(frame_pil, text, (50, y_offset + i*25), color, font_small)

        # Логіка утримання
        current_time = time.time()
        if correct:
            if self.hold_start_time is None:
                self.hold_start_time = current_time
                # Відтворюємо звук "правильно"
                if sound_correct and (current_time - self.last_sound_time > self.sound_cooldown):
                    sound_correct.play()
                    self.last_sound_time = current_time
            else:
                elapsed = current_time - self.hold_start_time
```

```
                remaining = max(0, duration - elapsed)
                draw_text_pil(frame_pil, f"Утримуйте: {remaining:.1f} сек", (50, y_offset +
len(angle_specs)*25 + 10), COLOR_CORRECT, font)
                if elapsed >= duration:
                    # Крок виконано
                    if sound_step_done:
                        sound_step_done.play()
                    self.current_step += 1
                    self.hold_start_time = None
                    if self.current_step >= self.total_steps:
                        draw_text_pil(frame_pil, "Вітаємо! Вправа завершена.", (50, 300),
COLOR_CORRECT, font)
        else:
            self.hold_start_time = None
            draw_text_pil(frame_pil, "Поза неправильна", (50, y_offset + len(angle_specs)*25 +
10), COLOR_INCORRECT, font)

    def draw_trainer_mode(self, frame_pil, draw, landmarks):
        """Відображає інформацію в режимі тренера."""
        if self.current_step >= self.total_steps:
            draw_text_pil(frame_pil, "Усі кроки записано. Натисніть Save JSON.", (50, 50),
COLOR_CORRECT, font)
            return

        step = self.steps[self.current_step]
        step_num = step["stepNumber"]
        desc = step["description"]
        body_part = step["bodyPart"]
        angle_specs = step["angles"]

        draw_text_pil(frame_pil, f"Режим: Тренер", (50, 20), COLOR_TEXT, font_small)
        draw_text_pil(frame_pil, f"Крок {step_num}/{self.total_steps}: {desc}", (50, 50),
COLOR_TEXT, font)
        draw_text_pil(frame_pil, f"Частина тіла: {body_part}", (50, 80), COLOR_TEXT, font_small)

        if landmarks is None:
            draw_text_pil(frame_pil, "Тіло не видно", (50, 120), COLOR_INCORRECT, font)
            return

        # Обчислюємо поточні кути
        angles = get_current_angles(landmarks, angle_specs)

        # Відображаємо кути та поточні значення
        y_offset = 120
        for i, spec in enumerate(angle_specs):
            if angles[i] is not None:
                text = f"{spec['base_joint']}: {angles[i]:.1f}° (записано:
{spec.get('min_angle',0)}-{spec.get('max_angle',0)})"
            else:
                text = f"{spec['base_joint']}: недоступно"
            draw_text_pil(frame_pil, text, (50, y_offset + i*25), COLOR_TEXT, font_small)
```

```
        draw_text_pil(frame_pil, "Наведіть палець на кнопку RECORD STEP для запису", (50, y_offset
+ len(angle_specs)*25 + 20), COLOR_TEXT, font_small)

    def record_current_step(self):
        """Записує кути для поточного кроку в режимі тренера."""
        #  Тут  ми  не  маємо  доступу  до  landmarks,  тому  цей  метод  викликається  з
handle_finger_interaction,
        # але landmarks там немає. Треба передати landmarks через self.
        # Для спрощення зробимо так: в handle_finger_interaction зберігаємо останні landmarks у
змінну.
        # Додамо атрибут self.last_landmarks.
        if not hasattr(self, 'last_landmarks') or self.last_landmarks is None:
            print("Немає даних про позу")
            return

        step = self.steps[self.current_step]
        angle_specs = step["angles"]
        angles = get_current_angles(self.last_landmarks, angle_specs)

        for i, spec in enumerate(angle_specs):
            if angles[i] is not None:
                # Встановлюємо діапазон з запасом
                min_angle = max(0, angles[i] - MARGIN_DEG)
                max_angle = min(180, angles[i] + MARGIN_DEG)
                spec["min_angle"] = round(min_angle, 1)
                spec["max_angle"] = round(max_angle, 1)
                print(f"Записано    кут    {angles[i]:.1f}°    ->    діапазон    {min_angle:.1f}-
{max_angle:.1f}")
            else:
                print(f"Не вдалося обчислити кут для {spec['base_joint']}")

        # Можна автоматично перейти до наступного кроку
        if self.current_step < self.total_steps - 1:
            self.current_step += 1
        else:
            print("Усі кроки записано")

# ==================== Точка входу ====================
if __name__ == "__main__":
    if len(sys.argv) > 1:
        json_file = sys.argv[1]
    else:
        json_file = "warrior2.json"  # за замовчуванням
        # Для тесту можна створити приклад JSON з наданими даними

    app = YogaTrainerApp(json_file)
    app.run()
```

ДОДАТОК 2 – ПОВНА JSON-СТРУКТУРА ВПРАВ ДЛЯ ПРОГРАМИ


```
{"YogaPose": {
    "name": {"type": "string", "description": "Назва вправи (українською)"},
    "sanskrit_name": {"type": "string", "description": "Назва вправи (санскритом)"},
    "description": {"type": "string", "description": "Повний текстовий опис змісту вправи та методики її виконання."},
    "difficulty_level": {"type": "string", "description": "Рівень складності (напр. початковий/середній/високий)"},
    "category": {"type": "string", "description": "Категорія вправи (напр. стоячи, сидячи, і т.д.)"},
    "asana_details": {
      "breathing": {"type": "string", "description": "Опис техніки дихання під час виконання асани."},
      "maintained_without_tension": {"type": "string", "description": "Інструкції щодо утримання пози без зайвого напруження."},
      "focus_stability_balance": {"type": "string", "description": "Опис точок концентрації та методів збереження рівноваги."}},
    "steps": [ {
        "stepNumber": {"type": "integer", "description": "Порядковий номер кроку виконання."},
        "description": {"type": "string", "description": "Текстовий опис дій на конкретному кроці."},
        "duration": {"type": "integer", "description": "Час затримки пози у секундах."},
        "bodyPart": {"type": "string", "description": "Назва частини тіла, на якій акцентується увага."},
        "angles": [
          { "joint1": {"type": "string", "description": "Назва 1-го суглоба за MediaPipe."},
            "base_joint": {"type": "string", "description": "Назва опорного суглоба-вершини."},
            "joint2": {"type": "string", "description": "Назва 2-го суглоба за MediaPipe."},
            "min_angle": {"type": "number", "description": "Мінімальний поріг кута у градусах."},
            "max_angle": {"type": "number", "description": "Максимальний поріг кута у градусах."}
} ] } ],
    "modifications": [
      { "variations": {"type": "string", "description": "Опис варіацій виконання вправи."},
        "adaptationOptions": {"type": "string", "description": "Варіанти адаптації під фізичні особливості."} } ],
    "commonMistakes": [
      { "frequentErrors": {"type": "string", "description": "Перелік типових помилок користувачів."},
        "incorrectTechniques": {"type": "string", "description": "Опис технічно неправильного виконання."} } ],
    "safetyMeasures": [
      { "precautions": {"type": "string", "description": "Заходи безпеки під час виконання."},
        "contraindications": {"type": "string", "description": "Медичні протипоказання."},
        "warnings": {"type": "string", "description": "Критичні застереження щодо травм."} } ],
    "equipment": [
      { "name": {"type": "string", "description": "Назва необхідного інвентарю."},
        "usage": {"type": "string", "description": "Опис використання інвентарю."} } ],
    "benefits": [
      { "positiveEffects": {"type": "string", "description": "Загальний позитивний вплив на організм."},
```

"physicalBenefits": {"type": "string", "description": "Користь для фізичного здоров'я."},
"mentalBenefits": {"type": "string", "description": "Користь для ментального стану."} }
] } }

ДОДАТОК 3 – ПОВНА ЗАПОВНЕНА ДЛЯ ВПРАВИ «ВОЇН II» СТРУКТУРА


```
{"YogaPose": {
    "name": "Поза воїна II (Warrior II Pose)",
     "sanskrit_name": "Virabhadrasana II",
    "description": "У цій позі переднє коліно згинається, створюючи розтяжку стегон, ваші руки витягуються в обидва боки, погляд залишається спокійним і стійким у напрямку передньої руки.",
    "difficulty_level": "Beginner/Intermediate",
    "category": "Standing",
    "asana_details": {
      "breathing": "Зробіть від 5 до 10 циклів дихання (приблизно 40 секунд).",
      "maintained_without_tension": "Усвідомте дискомфорт, тримайте м'язи обличчя розслабленими.",
      "focus_stability_balance": "Погляд поверх передньої руки."},
    "steps": [ {
        "stepNumber": 1,
        "description": "З «Пози собаки» крок правою ногою вперед.",
        "duration": 3,
            "bodyPart": "Права нога",
        "angles": [ {
            "joint1": "RIGHT_HIP",
            "base_joint": "RIGHT_KNEE",
            "joint2": "RIGHT_ANKLE",
            "min_angle": 120.0,
            "max_angle": 170.0
          } ] },
      { "stepNumber": 2,
        "description": "Зігніть праве коліно на 90 градусів.",
        "duration": 5,
            "bodyPart": "Права нога",
        "angles": [ {
            "joint1": "RIGHT_HIP",
            "base_joint": "RIGHT_KNEE",
            "joint2": "RIGHT_ANKLE",
            "min_angle": 75.0,
            "max_angle": 115.0
          } ] },
      { "stepNumber": 3,
        "description": "Опустіть ліву п’яту, стопа під 90 градусів.",
        "duration": 4,
            "bodyPart": "Ліва нога",
        "angles": [ {
            "joint1": "LEFT_HIP",
            "base_joint": "LEFT_ANKLE",
            "joint2": "LEFT_FOOT_INDEX",
            "min_angle": 75.0,
            "max_angle": 105.0
          } ] },
      { "stepNumber": 4,
        "description": "Розподіліть масу тіла рівномірно.",
        "duration": 3,
```

```
        "bodyPart": "Ноги",
  "angles": [ {
      "joint1": "LEFT_ANKLE",
      "base_joint": "LEFT_HIP",
      "joint2": "RIGHT_HIP",
      "min_angle": 75.0,
      "max_angle": 115.0
    } ] },
{ "stepNumber": 5,
  "description": "На вдиху підніміть тулуб і руки вверх.",
  "duration": 3,
      "bodyPart": "Руки",
  "angles": [ {
      "joint1": "LEFT_HIP",
      "base_joint": "LEFT_SHOULDER",
      "joint2": "LEFT_ELBOW",
      "min_angle": 75.0,
      "max_angle": 105.0
    } ] },
{ "stepNumber": 6,
  "description": "Витягніть руки літерою Т.",
  "duration": 3,
      "bodyPart": "Руки",
  "angles": [ {
      "joint1": "LEFT_WRIST",
      "base_joint": "LEFT_SHOULDER",
      "joint2": "RIGHT_SHOULDER",
      "min_angle": 160.0,
      "max_angle": 200.0
    } ] },
{ "stepNumber": 7,
  "description": "Руки паралельно підлозі, долоні вниз.",
  "duration": 3,
      "bodyPart": "Руки",
  "angles": [ {
      "joint1": "RIGHT_SHOULDER",
      "base_joint": "RIGHT_ELBOW",
      "joint2": "RIGHT_WRIST",
      "min_angle": 160.0,
      "max_angle": 200.0
    } ] },
{ "stepNumber": 8,
  "description": "Поверніть голову вперед.",
  "duration": 2,
      "bodyPart": "Голова",
  "angles": [ {
      "joint1": "LEFT_SHOULDER",
      "base_joint": "NOSE",
      "joint2": "RIGHT_SHOULDER",
      "min_angle": 70.0,
      "max_angle": 110.0
    } ] },
```

```
  { "stepNumber": 9,
    "description": "Стегна повернуті назовні.",
    "duration": 2,
        "bodyPart": "Стегна",
    "angles": [ {
        "joint1": "LEFT_KNEE",
        "base_joint": "LEFT_HIP",
        "joint2": "RIGHT_HIP",
        "min_angle": 150.0,
        "max_angle": 190.0
      } ] },
  { "stepNumber": 10,
    "description": "Утримання пози (статична фаза).",
    "duration": 40,
        "bodyPart": "Усе тіло",
    "angles": [ {
        "joint1": "RIGHT_HIP",
        "base_joint": "RIGHT_KNEE",
        "joint2": "RIGHT_ANKLE",
        "min_angle": 60.0,
        "max_angle": 130.0
      }, {
        "joint1": "LEFT_WRIST",
        "base_joint": "LEFT_SHOULDER",
        "joint2": "RIGHT_WRIST",
        "min_angle": 160.0,
        "max_angle": 200.0
      } ] },
  { "stepNumber": 11,
    "description": "Повернення в позу собаки.",
    "duration": 5,
        "bodyPart": "Права нога",
    "angles": [ {
        "joint1": "RIGHT_SHOULDER",
        "base_joint": "RIGHT_HIP",
        "joint2": "RIGHT_KNEE",
        "min_angle": 40.0,
        "max_angle": 75.0
      } ] } ],
"modifications": [ {
    "variations": "Виконання на стільці; менш глибокий випад.",
    "adaptationOptions": "Зменшення дистанції між ногами для початківців."} ],
"commonMistakes": [ {
    "frequentErrors": "Коліно завалюється всередину, опускання рук.",
    "incorrectTechniques": "Стегна спрямовані вперед, а не вбік."} ],
"safetyMeasures": [ {
    "precautions": "Тримайте шию в нейтральному положенні при болях.",
    "contraindications": "Травми колін, стегон або спини.",
        "warnings": "Будьте уважні при болях у шиї."} ],
 "equipment": [
  { "name": "Назва необхідного інвентарю (напр. блок, ремінь)",
    "usage": "Опис того, як саме використовувати цей інвентар у вправі."} ],
```

```
   "benefits": [ {
       "positiveEffects": "Зміцнює все тіло, покращує поставу.",
       "physicalBenefits": "Зміцнює ядро, ноги та розтягує грудну клітку.",
       "mentalBenefits": "Розвиває концентрацію та витривалість."
} ] } }
```